

CONDITIONALS & LOOPS

Dynamic Task Execution and
Bulk Operations in Ansible

Professional Reference Documentation

Version 1.0

Table of Contents

- 1. Introduction
- 2. Conditionals with when
 - 2.1 Basic Syntax
 - 2.2 Conditional Operators
 - 2.3 Conditions on OS and Distribution
 - 2.4 Conditions on Variable Values
 - 2.5 Conditions on Previous Task Results
 - 2.6 Combining Conditions
 - 2.7 Conditional Imports and Includes
- 3. Loops
 - 3.1 The loop Keyword
 - 3.2 Iterating over Dictionaries
 - 3.3 Registering Loop Results
 - 3.4 Loop Control Options
- 4. Legacy with_* Loops
 - 4.1 with_items
 - 4.2 with_dict
 - 4.3 with_fileglob
 - 4.4 Other with_* Variants
 - 4.5 Migration to loop
- 5. Practical Patterns
 - 5.1 Bulk User Creation
 - 5.2 Bulk Package Installation
 - 5.3 Templating Multiple Files
 - 5.4 Combining Loops and Conditionals
- 6. Best Practices
- 7. Quick Reference

1. Introduction

Static playbooks that always execute the same tasks in the same order are useful, but real infrastructure is rarely uniform. Servers run different operating systems, hosts hold different roles, and configuration often depends on the current state of a system. Ansible provides two features that make playbooks adaptive and concise:

- **Conditionals** — the `when` keyword controls whether a task runs, based on the operating system, a variable value, the result of a previous task, or any combination of these.
- **Loops** — the `loop` keyword and the legacy `with_*` constructs iterate a task over a list or dictionary, eliminating repetitive task definitions.

Together they allow a single playbook to manage a heterogeneous fleet with clear, readable logic. This document covers both features in depth and closes with practical patterns for bulk user creation, package installation, and file templating.

2. Conditionals with when

2.1 Basic Syntax

The `when` keyword accepts a Jinja2 expression. If the expression evaluates to true, the task runs; otherwise it is skipped. Unlike the `{{ }}` syntax used elsewhere, the `when` value is already a Jinja2 context, so braces are *not* used.

```
- name: Install a package only on Debian systems
  ansible.builtin.package:
    name: apache2
    state: present
  when: ansible_facts.os_family == "Debian"
```

Common mistake: writing `when: "{{ ansible_facts.os_family }}" == 'Debian'`. The correct form omits the outer braces entirely.

2.2 Conditional Operators

Ansible supports the full Jinja2 / Python expression language, including:

Operator	Meaning	Example
<code>== / !=</code>	Equality / inequality.	<code>os_family == "Debian"</code>
<code>> / <</code>	Numeric comparison.	<code>memtotal_mb > 4096</code>
<code>>= / <=</code>	Inclusive comparison.	<code>processor_vcpus >= 8</code>
<code>and / or</code>	Logical conjunction / disjunction.	<code>a and (b or c)</code>
<code>not</code>	Logical negation.	<code>not debug_enabled</code>
<code>in / not in</code>	Membership test.	<code>"nginx" in packages</code>
<code>is defined</code>	Variable exists.	<code>app_port is defined</code>

Operator	Meaning	Example
is undefined	Variable missing.	app_port is undefined
is file / is directory	Path type test.	path is directory
truthy	Value is true.	debug_enabled bool

Table 1 — Common operators in when expressions.

2.3 Conditions on OS and Distribution

Operating-system conditions are among the most common uses of when. Facts provide `os_family`, `distribution`, `distribution_version`, and `distribution_major_version`.

```
- name: Install the Debian web server
  ansible.builtin.package:
    name: apache2
    state: present
  when: ansible_facts.os_family == "Debian"

- name: Install the Red Hat web server
  ansible.builtin.package:
    name: httpd
    state: present
  when: ansible_facts.os_family == "RedHat"

- name: Apply a workaround only on Ubuntu 22.04
  ansible.builtin.copy:
    src: workaround.conf
    dest: /etc/myapp/workaround.conf
  when:
    - ansible_facts.distribution == "Ubuntu"
    - ansible_facts.distribution_version is version("22.04", ">=")

- name: Apply a package only on Debian-family hosts
  ansible.builtin.package:
    name: apt-transport-https
    state: present
  when: ansible_facts.os_family == "Debian"
```

Tip: The `version()` test compares version strings semantically, which is safer than string comparison for values such as "9" versus "10".

2.4 Conditions on Variable Values

Conditions on variables let a playbook behave differently based on environment, role, or explicit user input.

```
# Enable debug logging only in staging
- name: Enable debug logging
  ansible.builtin.lineinfile:
    path: /etc/myapp/app.conf
    line: "log_level=debug"
    when: environment == "staging"

# Install a package only if the feature flag is true
- name: Install monitoring agent
  ansible.builtin.package:
    name: node_exporter
    state: present
    when: monitoring_enabled | bool

# Take action only if the variable is defined
- name: Configure proxy
  ansible.builtin.template:
    src: proxy.conf.j2
    dest: /etc/myapp/proxy.conf
    when: proxy_url is defined

# Check a value against a list
- name: Install only supported versions
  ansible.builtin.package:
    name: "myapp-{{ app_version }}"
    state: present
    when: app_version in supported_versions
```

2.5 Conditions on Previous Task Results

The `register` keyword captures the result of a task into a variable. Subsequent tasks can inspect that variable in a `when` condition. This is the standard way to implement "only run this if the previous step changed something".

```
- name: Check whether the application is installed
  ansible.builtin.stat:
    path: /opt/myapp/bin/myapp
    register: myapp_binary

- name: Install the application
  ansible.builtin.unarchive:
    src: myapp.tar.gz
    dest: /opt/myapp
    when: not myapp_binary.stat.exists

- name: Run the database migration
  ansible.builtin.command: /opt/myapp/bin/migrate
  register: migration
  changed_when: migration.rc == 0

- name: Restart the application if the migration changed something
  ansible.builtin.service:
    name: myapp
    state: restarted
    when: migration.changed
```

Common attributes on a registered result that are useful in conditions:

- `result.changed` — true if the task modified the system.
- `result.failed` — true if the task failed.
- `result.rc` — exit code for command-like modules.
- `result.stdout` / `result.stderr` — captured output.
- `result.stat` — file metadata when using the `stat` module.

2.6 Combining Conditions

A `when` value may be a list. All items in the list must be true (logical AND). Use a single string with `and` / `or` when more complex logic is required.

```
# AND -- all conditions must be true (recommended form)
- name: Install the package on Debian 22.04 or later
  ansible.builtin.package:
    name: myapp
    state: present
  when:
    - ansible_facts.os_family == "Debian"
    - ansible_facts.distribution_version is version("22.04", ">=")
    - install_myapp | bool

# OR -- any condition may be true (single expression)
- name: Restart the service when either file changed
  ansible.builtin.service:
    name: myapp
    state: restarted
  when: config_changed or binary_changed

# Complex -- grouped logic inside a single string
- name: Complex condition
  ansible.builtin.debug:
    msg: "Condition met"
  when: >
    (ansible_facts.os_family == "Debian" and env == "production")
    or (ansible_facts.os_family == "RedHat" and env == "staging")
```

2.7 Conditional Imports and Includes

The `when` keyword may be applied to `include_tasks` and `import_tasks` to conditionally load an entire file of tasks.

```
- name: Apply Debian-specific configuration
  ansible.builtin.include_tasks: debian.yml
  when: ansible_facts.os_family == "Debian"

- name: Apply Red Hat-specific configuration
  ansible.builtin.include_tasks: redhat.yml
  when: ansible_facts.os_family == "RedHat"
```

3. Loops

Loops allow a single task to be executed multiple times with different input values. They are the primary tool for bulk operations: creating many users, installing many packages, or templating many files from one task definition.

3.1 The loop Keyword

loop is the modern, recommended way to iterate. It accepts a list of values, and each iteration exposes the current value as `{{ item }}`.

```
# Install multiple packages
- name: Install common utilities
  ansible.builtin.package:
    name: "{{ item }}"
    state: present
  loop:
    - git
    - curl
    - vim
    - htop

# Create multiple directories
- name: Create application directories
  ansible.builtin.file:
    path: "{{ item }}"
    state: directory
    owner: deploy
    group: deploy
    mode: '0755'
  loop:
    - /opt/myapp
    - /opt/myapp/releases
    - /opt/myapp/shared
    - /opt/myapp/logs

# Iterate over a variable holding a list
- name: Create users listed in a variable
  ansible.builtin.user:
    name: "{{ item }}"
    state: present
  loop: "{{ app_users }}"
```

Note: When loop receives a string instead of a list, Ansible treats it as a single-element list. To iterate over a comma-separated string, split it first with the `split()` filter: `loop: "{{ csv_string.split(',') }}"`.

3.2 Iterating over Dictionaries

Two patterns are common when iterating over dictionaries. Use the `dict2items` filter to turn a dictionary into a list of key/value pairs, or use `items2dict` for the reverse.

```
# vars:
#   users:
#     alice:
#       uid: 2001
#       shell: /bin/bash
#     bob:
#       uid: 2002
#       shell: /bin/zsh

- name: Create users from a dictionary
  ansible.builtin.user:
    name: "{{ item.key }}"
    uid: "{{ item.value.uid }}"
    shell: "{{ item.value.shell }}"
    state: present
    loop: "{{ users | dict2items }}"
```

Inside the loop, the current entry is a dictionary with two keys: `key` and `value`. The `value` may itself be a nested dictionary, as shown above.

3.3 Registering Loop Results

When a task that uses a loop is registered, the result is a list — one entry per iteration — stored under the `results` key. You can inspect individual iterations or filter them downstream.

```
- name: Check whether each config file exists
  ansible.builtin.stat:
    path: "/etc/myapp/{{ item }}.conf"
  loop:
    - app
    - db
    - cache
  register: config_files

- name: Show which config files are missing
  ansible.builtin.debug:
    msg: "Missing: {{ item.item }}"
  loop: "{{ config_files.results }}"
  when: not item.stat.exists
```

3.4 Loop Control Options

Ansible provides several keywords that change how loops behave, especially regarding change reporting and variable scoping.

Keyword	Purpose
<code>loop_control.label</code>	Shorten the output shown for each iteration.
<code>loop_control.pause</code>	Pause N seconds between iterations.
<code>loop_control.index_var</code>	Expose the current index under a custom name.
<code>loop_control.loop_var</code>	Rename "item" to something more meaningful.
<code>loop_control.extended</code>	Provide <code>ansible_loop</code> metadata (index, first, last, length).
<code>until / retries / delay</code>	Retry a task until it succeeds or the retry count is exhausted.

Keyword	Purpose
changed_when	Override how change is determined for each iteration.

*Table 2 — Loop control options.***Example — clean output and custom variable names:**

```
- name: Create users
  ansible.builtin.user:
    name: "{{ user.name }}"
    groups: "{{ user.groups }}"
    state: present
  loop: "{{ app_users }}"
  loop_control:
    loop_var: user
    label: "{{ user.name }}"
    pause: 1 # wait one second between iterations
```

Example — retry a task until it succeeds:

```
- name: Wait for the API to become available
  ansible.builtin.uri:
    url: http://localhost:8080/health
    status_code: 200
  register: result
  until: result.status == 200
  retries: 10
  delay: 5
```

4. Legacy with_* Loops

Before the `loop` keyword was introduced, loops were expressed with the `with_*` family of keywords. They remain fully supported for backwards compatibility, and several of them still have no direct `loop` equivalent. Where a direct equivalent exists, `loop` with a filter is preferred for clarity and future-proofing.

4.1 with_items

The most common legacy loop. Equivalent to `loop` for simple lists. It also flattens nested lists by one level, which `loop` does not.

```
# Legacy form
- name: Install packages
  ansible.builtin.package:
    name: "{{ item }}"
    state: present
  with_items:
    - git
    - curl
    - vim

# Modern equivalent
- name: Install packages
  ansible.builtin.package:
    name: "{{ item }}"
    state: present
  loop:
    - git
    - curl
    - vim
```

4.2 with_dict

Iterates over a dictionary. Each iteration exposes `item.key` and `item.value`. The modern equivalent uses the `dict2items` filter with `loop`.

```
# Legacy form
- name: Create users from a dictionary
  ansible.builtin.user:
    name: "{{ item.key }}"
    uid: "{{ item.value.uid }}"
    state: present
  with_dict:
    alice:
      uid: 2001
    bob:
      uid: 2002

# Modern equivalent
- name: Create users from a dictionary
  ansible.builtin.user:
    name: "{{ item.key }}"
    uid: "{{ item.value.uid }}"
    state: present
  loop: "{{ users | dict2items }}"
  vars:
    users:
      alice:
        uid: 2001
      bob:
        uid: 2002
```

4.3 with_fileglob

Iterates over local files matching a glob pattern on the *control node*. This is the standard way to copy or template a directory of configuration files without listing each one. There is no direct loop equivalent; the `with_fileglob` keyword remains the idiomatic form.

```
- name: Deploy all nginx site configurations
  ansible.builtin.copy:
    src: "{{ item }}"
    dest: /etc/nginx/sites-available/
    owner: root
    group: root
    mode: '0644'
  with_fileglob:
    - files/nginx/sites/*.conf
  notify: Reload nginx

- name: Deploy all application templates
  ansible.builtin.template:
    src: "{{ item }}"
    dest: "/etc/myapp/{{ item | basename | regex_replace('\.j2$', '') }}"
    owner: deploy
    group: deploy
    mode: '0640'
  with_fileglob:
    - templates/configs/*.conf.j2
```

4.4 Other with_* Variants

Keyword	Iterates over	Modern equivalent
<code>with_items</code>	A simple list.	<code>loop</code>
<code>with_dict</code>	A dictionary.	<code>loop + dict2items</code>
<code>with_fileglob</code>	Local files matching a glob.	(no direct equivalent)
<code>with_file</code>	The contents of local files.	<code>loop + lookup("file", ...)</code>
<code>with_lines</code>	Lines from a command's output.	<code>loop + lookup("pipe", ...)</code>
<code>with_sequence</code>	A numeric sequence (start, end, step).	<code>loop + range()</code>
<code>with_nested</code>	Cartesian product of lists.	<code>loop + product filter</code>
<code>with_together</code>	Parallel lists (zip).	<code>loop + zip filter</code>
<code>with_subelements</code>	Nested sublists.	<code>loop + subelements filter</code>
<code>with_random_choice</code>	A random item from a list.	<code>loop + random filter</code>
<code>with_inventory_hostnames</code>	Hostnames from a group.	<code>loop + groups[...]</code>
<code>with_first_found</code>	The first existing file.	<code>loop + first_found lookup</code>

Table 3 — Legacy `with_*` loops and their modern replacements.

Examples of the more useful legacy forms:

```
# with_sequence -- create numbered directories
- name: Create numbered log directories
  ansible.builtin.file:
    path: "/var/log/myapp/{{ item }}"
    state: directory
    mode: '0755'
  with_sequence: start=1 end=5 stride=1 format=%02d

# with_file -- read file contents
- name: Display the contents of several files
  ansible.builtin.debug:
    msg: "{{ item }}"
  with_file:
    - files/motd.txt
    - files/issue.txt

# with_nested -- cartesian product
- name: Create user home directories for each environment
  ansible.builtin.file:
    path: "/home/{{ item[0] }}/{{ item[1] }}"
    state: directory
  with_nested:
    - ['alice', 'bob']
    - ['projects', 'backups']
```

4.5 Migration to loop

Because `loop` is the supported modern form, prefer it for any new playbook. The following table summarises the mechanical transformations.

```
# with_items -> loop
loop: "{{ my_list }}"

# with_dict -> loop + dict2items
loop: "{{ my_dict | dict2items }}"

# with_sequence -> loop + range
loop: "{{ range(1, 6) | list }}"

# with_nested -> loop + product
loop: "{{ list_a | product(list_b) | list }}"

# with_together -> loop + zip
loop: "{{ list_a | zip(list_b) | list }}"

# with_subelements -> loop + subelements
loop: "{{ users | subelements('keys') }}"

# with_fileglob -> no direct equivalent; keep with_fileglob
# with_first_found -> no direct equivalent; keep with_first_found
```

5. Practical Patterns

5.1 Bulk User Creation

The combination of a dictionary variable and loop with `dict2items` produces clean, data-driven user management.

```
# group_vars/all.yml
managed_users:
  alice:
    uid: 2001
    shell: /bin/bash
    groups: [sudo, docker]
    ssh_key: files/keys/alice.pub
  bob:
    uid: 2002
    shell: /bin/zsh
    groups: [developers]
    ssh_key: files/keys/bob.pub
  carol:
    uid: 2003
    shell: /bin/bash
    groups: [sudo]
    ssh_key: files/keys/carol.pub

# site.yml
- name: Provision standard users
  hosts: all
  become: true
  tasks:
    - name: Create each managed user
      ansible.builtin.user:
        name: "{{ item.key }}"
        uid: "{{ item.value.uid }}"
        shell: "{{ item.value.shell }}"
        groups: "{{ item.value.groups }}"
        append: true
        create_home: true
        state: present
      loop: "{{ managed_users | dict2items }}"
      loop_control:
        label: "{{ item.key }}"

    - name: Install each user's SSH key
      ansible.posix.authorized_key:
        user: "{{ item.key }}"
        state: present
        key: "{{ lookup('file', item.value.ssh_key) }}"
      loop: "{{ managed_users | dict2items }}"
      loop_control:
        label: "{{ item.key }}"
```

5.2 Bulk Package Installation

Installing many packages is the classic use of `loop`. For simple lists, the `package` module already accepts a list, which is even faster because it makes a single transaction. Use an explicit loop when each package

needs different arguments or conditions.

```
# Faster -- pass the whole list at once
- name: Install base packages
  ansible.builtin.package:
    name:
      - git
      - curl
      - vim
      - htop
      - jq
      - net-tools
    state: present

# Use a loop when conditionals or per-item data are needed
- name: Install packages conditionally by OS family
  ansible.builtin.package:
    name: "{{ item.name }}"
    state: present
  loop:
    - { name: apache2,      family: Debian }
    - { name: httpd,       family: RedHat }
    - { name: firewalld,   family: RedHat }
    - { name: ufw,         family: Debian }
  when: item.family == ansible_facts.os_family

# Retry each package install until success
- name: Install packages with retries
  ansible.builtin.package:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
    - postgresql
    - redis
  register: pkg_result
  until: pkg_result is succeeded
```

```
retries: 3
delay: 5
```

5.3 Templating Multiple Files

Rendering a directory of Jinja2 templates with different destinations is a common deployment pattern. Two approaches are idiomatic.

Approach 1 — iterate over a list of file descriptors:

```
- name: Render application configuration files
  ansible.builtin.template:
    src: "{{ item.src }}"
    dest: "{{ item.dest }}"
    owner: deploy
    group: deploy
    mode: "{{ item.mode | default('0644') }}"
  loop:
    - { src: app.conf.j2,      dest: /etc/myapp/app.conf }
    - { src: db.conf.j2,      dest: /etc/myapp/db.conf }
    - { src: cache.conf.j2,   dest: /etc/myapp/cache.conf, mode: '0640' }
  notify: Restart myapp
```

Approach 2 — use with_fileglob to render every template:

```
- name: Render every template in the configuration directory
  ansible.builtin.template:
    src: "{{ item }}"
    dest: "/etc/myapp/{{ item | basename | regex_replace('\.j2$', '') }}"
    owner: deploy
    group: deploy
    mode: '0644'
  with_fileglob:
    - templates/configs/*.conf.j2
  notify: Restart myapp
```

5.4 Combining Loops and Conditionals

Loops and conditionals compose naturally. Inside a loop, the when expression can reference item, so each iteration can be evaluated independently.

```
- name: Create only the users whose accounts are enabled
  ansible.builtin.user:
    name: "{{ item.key }}"
    state: present
  loop: "{{ managed_users | dict2items }}"
  when: item.value.enabled | default(true) | bool

- name: Apply firewall rules only on the production environment
  ansible.builtin.ufw:
    rule: allow
    port: "{{ item }}"
    proto: tcp
  loop: [22, 80, 443, 8080]
  when: environment == "production"

- name: Restart each service that changed
  ansible.builtin.service:
    name: "{{ item.name }}"
    state: restarted
  loop: "{{ services }}"
  when: item.config_changed | bool
```

6. Best Practices

- **Prefer loop over with_items.** Use loop for any simple list; reserve with_* for the forms that have no modern equivalent (with_fileglob, with_first_found).
- **Do not use braces in when.** Write when: `x == "y"`, never when: `{{ x }} == 'y'`.
- **Prefer a list of conditions over "and".** A YAML list is readable and easy to extend.
- **Guard against undefined variables.** Use `is defined` or the `default()` filter.
- **Use loop_control.label.** It keeps output compact and readable when items are large dictionaries.
- **Rename item with loop_var.** A descriptive name such as `user` or `package` makes the task easier to read.
- **Prefer bulk arguments when possible.** Passing a list to `package` is faster than looping; use a loop only when per-item logic is required.
- **Keep loops small.** A loop with dozens of unrelated items is usually a sign the task should be split.
- **Use register + when for idempotent changes.** Check the current state before acting, rather than running a command unconditionally.
- **Add retries to network-dependent tasks.** `until`, `retries`, and `delay` make loops resilient to transient failures.

7. Quick Reference

Goal	Syntax
Condition on OS family	<code>when: ansible_facts.os_family == "Debian"</code>
Condition on a variable	<code>when: environment == "production"</code>
Guard against undefined	<code>when: my_var is defined</code>
Multiple AND conditions	<code>when:\n - a == 1\n - b == 2</code>
OR condition	<code>when: a == 1 or b == 2</code>
Condition on previous result	<code>when: result.changed</code>
Register a task result	<code>- stat: path=/x\n register: result</code>
Simple loop over a list	<code>loop: [a, b, c]</code>
Loop over a variable list	<code>loop: "{{ my_list }}"</code>
Loop over a dictionary	<code>loop: "{{ my_dict dict2items }}"</code>
Loop over files matching a glob	<code>with_fileglob: files/*.conf</code>
Rename the loop variable	<code>loop_control:\n loop_var: user</code>
Shorten loop output	<code>loop_control:\n label: "{{ item.name }}"</code>
Retry a task until success	<code>until: result.rc == 0\nretries: 5\ndelay: 3</code>
Pause between iterations	<code>loop_control:\n pause: 2</code>
Loop and condition together	<code>loop: ...\nwhen: item.enabled</code>

Table 4 — Conditionals and loops quick reference.

This document describes Ansible Core behaviour as of version 2.14 and later. Loop keywords and conditional operators may evolve between releases; always consult the official documentation for the version in use.